



```
In [1]: #Dataframe is a collection of series
```

```
In [3]: #PDF 2
import pandas as pd
import numpy as np
```

```
In [5]: #2.1 CREATING A SERIES
#Single Column -> Series
#Multiple Columns -> DataFrame
labels = ['a','b','c']

#ALL THE STRUCTURES BELOW CAN BE CONVERTED TO SERIES
my_list = [10,20,30]
arr = np.array([10,20,30])
d = {'a':10,'b':20,'c':30}
```

```
In [10]: #2.2Creating a series from a list
pd.Series(data=my_list,index=labels)
#Creating a series from an array
pd.Series(arr,labels)

#Creating from a dict
pd.Series(d)
```

```
Out[10]: a    10
         b    20
         c    30
         dtype: int64
```

```
In [12]: # Even functions (although unlikely that you will use this)
pd.Series([sum,print,len])

0    <built-in function sum>
1    <built-in function print>
2    <built-in function len>
dtype: object
```

```
In [19]: # The key to using a Series is understanding its index.
# Pandas makes use of these index names or
# numbers by allowing for fast look ups of information
# (works like a hash table or dictionary).

ser1 = pd.Series([1,2,3,4],index = ['USA', 'Germany','USSR', 'Japan'])
ser2 = pd.Series([1,2,5,4],index = ['USA', 'Germany','Italy', 'Japan'])
```

```
In [16]: print(f"ser1 {ser1}")
         print(f"ser2 {ser2}")
         print(f"ser1+2 {ser1+ser2}")
```

```

ser1 USA      1
Germany    2
USSR       3
Japan      4
dtype: int64
ser2 USA      1
Germany    2
Italy      5
Japan      4
dtype: int64
ser1 Germany  4.0
Italy        NaN
Japan        8.0
USA          2.0
USSR         NaN
dtype: float64

```

```

In [18]: #Operations on series
         ser1['USA']

```

```

Out[18]: 1

```

```

In [21]: #Whats the type of the object
         type(ser1)

```

```

Out[21]: pandas.core.series.Series

```

PDF 3 (DATAFRAMES)

DataFrames are the workhorse of pandas and are directly inspired by the R programming language. We can think of a DataFrame as a bunch of Series objects put together to share the same index

```

In [4]: from numpy.random import randn
         #Randn is useful for generating numbers from normal distribution
         #Whenever we want the same random number, we use seed.
         #Useful for tests
         print(np.random.seed(101))
         print(np.random.seed(102))

```

```

None

```

```

None

```

```

In [52]: df = pd.DataFrame(randn(5,4),index='A B C D E'.split(),columns='W X Y Z'.split()
         #.split would convert that sttring into 5 elements separated by " "
         type(df)
         df

```

```
Out[52]:
```

	W	X	Y	Z
A	0.854101	-1.307599	-1.485952	0.870746
B	0.575038	0.246979	-0.601873	-0.465131
C	0.183339	0.826680	-1.362105	-0.216016
D	0.206353	-1.641117	-1.666657	0.034567
E	-0.329217	0.936132	0.726568	0.646532

3.1 Selection and Indexing

Various methods to grab data from a DF

```
In [11]: #getting a single column
df.W #SQL SYNTAX (NOT RECOMMENDED)
df['W']

type(df.W) # PROOF that a single column is a series only
```

```
Out[11]: pandas.core.series.Series
```

```
In [13]: #getting multiple columns
x = df[
    ['W', 'Z']
]
type(x) #Atleast 2 cols is a Dataframe
```

```
Out[13]: pandas.core.frame.DataFrame
```

3.2 Creating a new column

```
In [14]: df['new'] = df['W'] + df['Y']
#Here we created a new column and appended to df, by simply adding 2 columns
df
```

```
Out[14]:
```

	W	X	Y	Z	new
A	1.447899	0.593138	-0.755616	-0.627166	0.692283
B	0.884035	0.162718	-2.502813	0.410791	-1.618778
C	0.728750	0.029140	-0.291052	0.314293	0.437698
D	0.997190	3.428563	-0.578567	-0.204163	0.418622
E	-1.867169	1.443434	0.403380	-0.690765	-1.463789

```
In [17]: #Removing a column
df.drop('new',axis=1) #axis =1 means delete by column
#
```

```
df# Note, not dropped in the original df. To do so we need to make inplace=True
```

Out[17]:

	W	X	Y	Z	new
A	1.447899	0.593138	-0.755616	-0.627166	0.692283
B	0.884035	0.162718	-2.502813	0.410791	-1.618778
C	0.728750	0.029140	-0.291052	0.314293	0.437698
D	0.997190	3.428563	-0.578567	-0.204163	0.418622
E	-1.867169	1.443434	0.403380	-0.690765	-1.463789

```
In [25]: df.drop('A',axis=0,inplace=True) #Now the deletion is in the original df
#Also, since we had axis=0, it can now delete from ABCDE(ROW)
df
```

KeyError

Traceback (most recent call last)

Cell In[25], line 1

```
----> 1 df.drop('A',axis=0,inplace=True) #Now the deletion is in the original d
f
      2 #Also, since we had axis=0, it can now delete from ABCDE(ROW)
      3 df
```

File ~\AppData\Roaming\Python\Python311\site-packages\pandas\core\frame.py:558
1, in DataFrame.drop(self, labels, axis, index, columns, level, inplace, error
s)

```
5433 def drop(
5434     self,
5435     labels: IndexLabel | None = None,
5436     (...)
5442     errors: IgnoreRaise = "raise",
5443 ) -> DataFrame | None:
5444     """
5445     Drop specified labels from rows or columns.
5446     (...)
5479         weight 1.0      0.8
5480     """
-> 5581     return super().drop(
5582         labels=labels,
5583         axis=axis,
5584         index=index,
5585         columns=columns,
5586         level=level,
5587         inplace=inplace,
5588         errors=errors,
5589     )
```

File ~\AppData\Roaming\Python\Python311\site-packages\pandas\core\generic.py:47
88, in NDFrame.drop(self, labels, axis, index, columns, level, inplace, errors)

```
4786 for axis, labels in axes.items():
4787     if labels is not None:
-> 4788         obj = obj._drop_axis(labels, axis, level=level, errors=errors)
4790 if inplace:
4791     self._update_inplace(obj)
```

File ~\AppData\Roaming\Python\Python311\site-packages\pandas\core\generic.py:48
30, in NDFrame._drop_axis(self, labels, axis, level, errors, only_slice)

```
4828 new_axis = axis.drop(labels, level=level, errors=errors)
4829 else:
-> 4830     new_axis = axis.drop(labels, errors=errors)
4831     indexer = axis.get_indexer(new_axis)
4833 # Case for non-unique axis
4834 else:
```

File ~\AppData\Roaming\Python\Python311\site-packages\pandas\core\indexes\base.py:7070, in Index.drop(self, labels, errors)

```
7068 if mask.any():
7069     if errors != "ignore":
```

```
-> 7070         raise KeyError(f"{labels[mask].tolist()} not found in axis")
    7071         indexer = indexer[~mask]
    7072     return self.delete(indexer)
```

KeyError: "['A'] not found in axis"

```
In [28]: #SELECTING ROWS
df.loc['E']
```

```
Out[28]: W    -0.983657
X      1.093005
Y    -1.251296
Z      1.481600
Name: E, dtype: float64
```

```
In [30]: df.iloc[2] #3rd row (starts from 0)
```

```
Out[30]: W    -1.279943
X      0.556753
Y    -0.048404
Z      0.234364
Name: D, dtype: float64
```

```
In [32]: df.loc['B','Y'] #Row B and Col Y
```

```
Out[32]: 1.0149926771961866
```

```
In [34]: df.loc[['E','B'],['W','Y']] #dataframe, 2 rows and 2 columns (2x2 matrix)
```

```
Out[34]:
```

	W	Y
E	-0.983657	-1.251296
B	1.553394	1.014993

3.4 Conditional Selection

```
In [37]: df>0 #Create a new df in boolean values
```

```
Out[37]:
```

	W	X	Y	Z
B	True	True	True	False
C	False	True	True	False
D	False	True	False	True
E	False	True	False	True

```
In [47]: df[df>0]
```

```
Out[47]:
```

	W	X	Y	Z
A	NaN	NaN	NaN	NaN
B	NaN	NaN	NaN	NaN
C	NaN	NaN	0.609151	1.038694
D	0.098843	NaN	NaN	1.017106
E	NaN	NaN	0.033912	0.424447

```
In [46]: df[df['W']>0]
```

```
Out[46]:
```

	W	X	Y	Z
D	0.098843	-1.109817	-0.754384	1.017106

```
In [53]: df[df['W']>0]['Y']
```

```
Out[53]: A    -1.485952
         B    -0.601873
         C    -1.362105
         D    -1.666657
         Name: Y, dtype: float64
```

```
In [58]: df[(df['W']>-1) & (df['Y'] > 0)]
```

```
Out[58]:
```

	W	X	Y	Z
E	-0.329217	0.936132	0.726568	0.646532

3.5 More index details

```
In [59]: df #DF with og index
```

```
Out[59]:
```

	W	X	Y	Z
A	0.854101	-1.307599	-1.485952	0.870746
B	0.575038	0.246979	-0.601873	-0.465131
C	0.183339	0.826680	-1.362105	-0.216016
D	0.206353	-1.641117	-1.666657	0.034567
E	-0.329217	0.936132	0.726568	0.646532

```
In [84]: df.reset_index()#NOT INPLACE
         df.reset_index()['index'] #This makes our older index into a new column, which
```

```
Out[84]: 0    A
         1    B
         2    C
         3    D
         4    E
         Name: index, dtype: object
```

```
In [94]: newind = 'CA CA WY OR CO'.split()
         df['States'] = newind
         df
```

```
Out[94]:
```

	W	X	Y	Z	States
States					
CA	0.854101	-1.307599	-1.485952	0.870746	CA
CN	0.575038	0.246979	-0.601873	-0.465131	CA
WY	0.183339	0.826680	-1.362105	-0.216016	WY
OR	0.206353	-1.641117	-1.666657	0.034567	OR
CO	-0.329217	0.936132	0.726568	0.646532	CO

```
In [95]: df.set_index('States')
```

```
Out[95]:
```

	W	X	Y	Z
States				
CA	0.854101	-1.307599	-1.485952	0.870746
CA	0.575038	0.246979	-0.601873	-0.465131
WY	0.183339	0.826680	-1.362105	-0.216016
OR	0.206353	-1.641117	-1.666657	0.034567
CO	-0.329217	0.936132	0.726568	0.646532

```
In [96]: df.set_index('States',inplace=True)
         df.loc['CA']
```

```
Out[96]:
```

	W	X	Y	Z
States				
CA	0.854101	-1.307599	-1.485952	0.870746
CA	0.575038	0.246979	-0.601873	-0.465131

3.5 Multi index Heirarchy (OPTIONAL)

```
In [98]: outside = ['G1','G1','G1','G2','G2','G2']
```



```

inside = [1,2,3,1,2,3]
hier_index = list(zip(outside,inside))
hier_index = pd.MultiIndex.from_tuples(hier_index)
hier_index

```

```

Out[98]: MultiIndex([('G1', 1),
                    ('G1', 2),
                    ('G1', 3),
                    ('G2', 1),
                    ('G2', 2),
                    ('G2', 3)],
                    )

```

PDF 4 MISSING DATA

```

In [101...] df = pd.DataFrame({'A':[1,2,np.nan],
                              'B':[5,np.nan,np.nan],
                              'C':[1,2,3]})
df #adding Nan values

```

```

Out[101...]

```

	A	B	C
0	1.0	5.0	1
1	2.0	NaN	2
2	NaN	NaN	3

```

In [105...] df.dropna() #this drops ALL the rows where even a SINGLE nan value exists
#DEFault axis is 0 (ROW)

```

```

Out[105...]

```

	A	B	C
0	1.0	5.0	1

```

In [104...] df.dropna(axis=1) #SAME LOGIC, but with COLUMNS

```

```

Out[104...]

```

	C
0	1
1	2
2	3

```

In [106...] df.dropna(thresh=2)#IF NUMBER OF NULL VALUES IN THAT ROW IS MORE THAN 2, ONLY
#THAT ROW

```

Out[106...

	A	B	C
0	1.0	5.0	1
1	2.0	NaN	2

In [107... `df.fillna('NEW VALUE')`

Out[107...

	A	B	C
0	1.0	5.0	1
1	2.0	NEW VALUE	2
2	NEW VALUE	NEW VALUE	3

In [109... `print(df['A'].mean())` *#null values would be replaced by this*
`df['A'].fillna(value=df['A'].mean())`
#Only take the A column
#fill its null values with the mean of that column

1.5

Out[109... `0 1.0`
`1 2.0`
`2 1.5`
Name: A, dtype: float64

In []: